คู่มือผู้นำเสนอ · Final Presentation

LabHub ระบบจัดการห้องแล็บ IoT

อ่านจบแล้วเล่าได้ว่าแอปทำอะไร มีอะไรบ้าง และพัฒนาด้วยอะไร โดยไม่ต้องรู้เรื่องโค้ด พร้อมบทพูด บทเดโมสด และคำตอบคำถามที่กรรมการน่าจะถาม

ฉบับวันที่ 29 ตุลาคม 2569 · ภาพหน้าจอทั้งหมดใช้ข้อมูลจำลอง (ชื่อคนและรหัสนักศึกษาเป็นของสมมติ)

วิธีใช้คู่มือนี้

อ่านส่วน 1–7 ก่อน
เข้าใจตัวแอปทั้งหมด ใช้เวลาประมาณ 40 นาที

กล่องสีส้ม "พูด"
ประโยคที่อ่านออกเสียงได้เลย ปรับเป็นคำพูดของตัวเองได้

ซ้อมเดโมด้วยส่วน 8
ซ้อมกับมือถือจริงอย่างน้อย 2 รอบก่อนวันนำเสนอ

สารบัญ

1. LabHub คืออะไร (เล่าใน 30 วินาที)
2. ใครใช้บ้าง: 3 บทบาท
3. ระบบยืม-คืนอุปกรณ์ (ฝั่งนักศึกษา)
4. งานของผู้ดูแล (TA / Admin)
5. ระบบห้องคอมพิวเตอร์
6. ใช้ได้บนอะไรบ้าง
7. พัฒนายังไง ใช้อะไร
8. บทเดโมสด
9. คำถามที่กรรมการน่าจะถาม
10. อ่านโค้ดสำหรับคนไม่เขียนโค้ด
11. คำศัพท์ที่อาจได้ยิน

แผนเวลา 25 นาที (แบ่ง 2 คน)

ช่วง	เวลา	ผู้พูด	ใช้ส่วนไหนของคู่มือ
เปิดตัว: ปัญหาเดิม + LabHub คืออะไร	3 นาที	คนที่ 1	ส่วน 1
ภาพรวม: 2 ระบบ 3 บทบาท	2 นาที	คนที่ 1	ส่วน 2
เดโมฝั่งนักศึกษา: ดูของ สแกนขอยืม ประวัติ	6 นาที	คนที่ 1 ถือมือถือ	ส่วน 3 + 8
เดโมฝั่งผู้ดูแล: อนุมัติคำขอ แดชบอร์ด	5 นาที	คนที่ 2 ใช้โน้ตบุ๊ก	ส่วน 4 + 8
ระบบห้องคอม: ผังห้อง แจ้งปัญหา งานซ่อม	4 นาที	คนที่ 2	ส่วน 5

ช่วง	เวลา	ผู้พูด	ใช้ส่วนไหนของคู่มือ
พัฒนายังไง ใช้เทคโนโลยีอะไร	4 นาที	คนที่ 1	ส่วน 6–7
สรุป + ถาม-ตอบ	1 นาที + ถามตอบ	ทั้งคู่	ส่วน 9

ถ้ามีเวลา 20 นาที: ตัดช่วงพัฒนาเหลือ 2 นาที และเดโมห้องคอมแค่ผังห้อง · ถ้ามี 30 นาที: เพิ่มเดโมนัดรับ (ส่วน 3) และรายงาน PDF (ส่วน 4)

1 LabHub คืออะไร

พูด **LabHub** คือแอปจัดการห้องแล็บ IoT ของคณะ ทำ 2 เรื่องในแอปเดียว เรื่องแรกคือ**ยืม-คืนอุปกรณ์ IoT** นักศึกษาสแกน QR ที่ตัวอุปกรณ์ ถ่ายรูป แล้วรอผู้ดูแลกดอนุมัติในแอป เรื่องที่สองคือ**ดูแลห้องคอมพิวเตอร์** ทุกคนเห็นว่าเครื่องไหนใช้ได้ เครื่องไหนเสีย และแจ้งปัญหาได้ทันที ใช้ได้ทั้งบนมือถือและบนเว็บ

ปัญหาก่อนมี LabHub ปรับให้ตรงกับที่ห้องแล็บเจอจริง

จดยืมด้วยมือ
สมุด/กระดาษ หายง่าย ค้นยาก ไม่รู้ว่าตอนนี้ใครถือของชิ้นไหนอยู่

ไม่รู้ว่าของว่างไหม
ต้องเดินมาถามที่ห้องก่อน ถึงจะรู้ว่ามีของให้ยืมหรือเปล่า

ตามของคืนยาก
ไม่มีใครเตือนเมื่อครบกำหนดคืน ของค้างอยู่กับคนยืมนาน

คอมเสียไม่มีใครรู้
นักศึกษาไม่มีช่องทางแจ้ง ผู้ดูแลไม่รู้ว่าเครื่องไหนต้องซ่อม

LabHub แก้ยังไง

ทุกชิ้นมี QR
สแกนแล้วรู้ทันทีว่าชิ้นไหน

ถ่ายรูปทุกครั้ง
มีหลักฐานสภาพตอนยืม-คืน

อนุมัติในแอป
ผู้ดูแลกดผ่านมือถือหรือคอม

เตือนอัตโนมัติ
ใกล้ครบกำหนด เกินกำหนด

ผังห้องคอมสด
เห็นเครื่องว่าง/เสีย แจ้งซ่อมได้

จำประโยคเดียว: "LabHub ทำให้การยืม-คืนอุปกรณ์และการดูแลห้องคอม มีหลักฐาน ตรวจสอบได้ และไม่ต้องใช้กระดาษ"

2 ใครใช้บ้าง: 3 บทบาท

ทุกคนเข้าระบบด้วย**บัญชี Google ของ มข.** (นักศึกษา @kkumail.com · อาจารย์/บุคลากร @kku.ac.th) เข้าครั้งแรกระบบสร้างบัญชีให้เอง ไม่ต้องสมัครสมาชิก

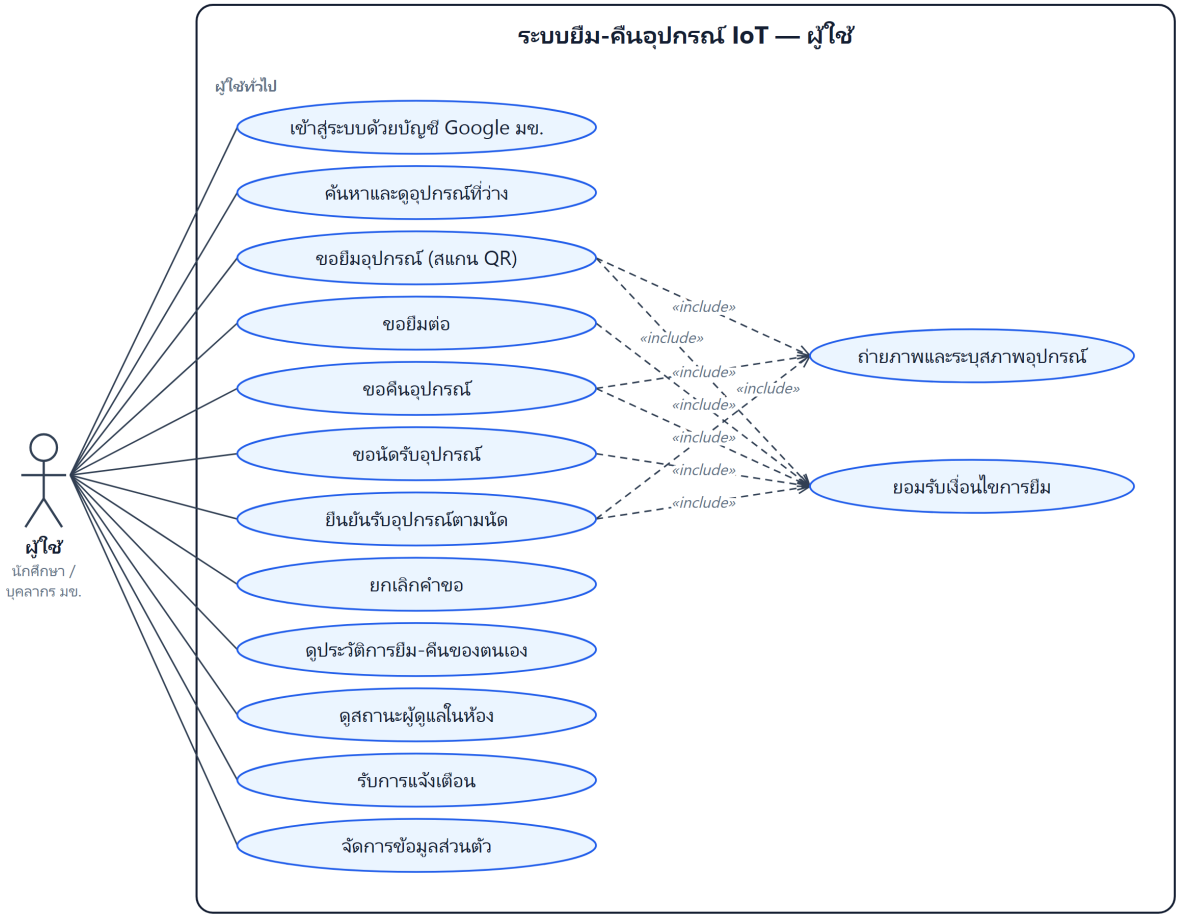
บทบาท	คือใคร	ทำอะไรได้
นักศึกษา (user)	ผู้ใช้ทั่วไป รวมถึงอาจารย์ที่ยังไม่ได้รับสิทธิ์	ดูของว่าง · ขอยืม/คืน/ยืมต่อ · นัดรับ · ดูประวัติของตัวเอง · ดูผังห้องคอม · แจ้งปัญหาเครื่อง

บทบาท	คือใคร	ทำอะไรได้
TA (ผู้ช่วย)	ผู้ช่วยดูแลห้อง ที่ Admin แต่งตั้ง	อนุมัติ/ปฏิเสธคำขอ · ตรวจสภาพตอนคืน · นัดเวลารับของ · ดูประวัติ/รายงาน · พิมพ์ป้าย QR · เปลี่ยนสถานะเครื่องคอม · งานซ่อม · ตรวจประจำเทอม
ผู้ดูแลระบบ (admin)	เจ้าของห้องแล็บ	ทุกอย่างของ TA + เพิ่ม/แก้/จำหน่ายอุปกรณ์ · หมวดหมู่ · นำเข้า CSV · ตั้งค่ากติกา · แต่งตั้ง TA/Admin · เพิ่มห้อง/เครื่อง/LAN

กติกาที่ควรจำ: TA อนุมัติคำขอ**ของตัวเอง**ไม่ได้ (กันอนุมัติให้ตัวเอง) · Admin เปลี่ยนสิทธิ์ของตัวเองไม่ได้ และระบบบังคับให้เหลือ Admin อย่างน้อย 1 คนเสมอ

พูด แอปเรามี 3 บทบาทครับ/ค่ะ นักศึกษาใช้ยืมของ TA เป็นผู้ช่วยที่อนุมัติคำขอได้ และ Admin จัดการได้ทุกอย่าง ทุกคนเข้าด้วยบัญชี Google ของมหาวิทยาลัย ไม่ต้องจำรหัสผ่านเพิ่ม

แผนภาพ Use Case (ใช้ประกอบถ้ากรรมการถามว่าใครทำอะไรได้)



อีก 2 แผนภาพ (ฝั่งผู้ดูแล และระบบห้องคอม) อยู่ที่ `docs/diagrams/` พร้อม ER Diagram และพจนานุกรมข้อมูลสำหรับรายงาน

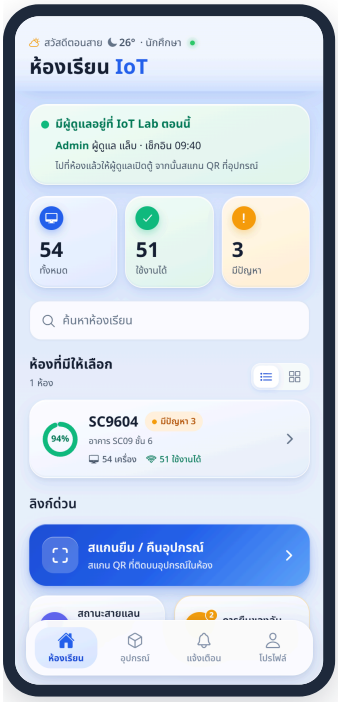
3 · ระบบยืม-คืนอุปกรณ์ (ฝั่งนักศึกษา)

หัวใจของแอป · ส่วนที่ควรเดโมสด

ขั้นตอนการยืม ตั้งแต่ต้นจนจบ

1. **ดูของว่าง** — แท็บอุปกรณ์
2. **สแกน QR** — ที่ตัวอุปกรณ์ในห้อง
3. **ถ่ายรูป + สภาพ** — เลือกจำนวนวันยืม
4. **ยอมรับกฎ** — แล้วกดส่งคำขอ
5. **ผู้ดูแลอนุมัติ** — ภายใน 30 นาที
6. **รับของ** — เริ่มนับวันยืม

ตอนคืนก็แบบเดียวกัน: สแกน QR ชิ้นที่ยืม → กด "ขอคืนอุปกรณ์" → ถ่ายรูป + บอกสภาพ → ผู้ดูแลตรวจรับ



หน้าแรก (ห้องเรียน)

หน้าแรก: เห็นทุกอย่างในจอเดียว

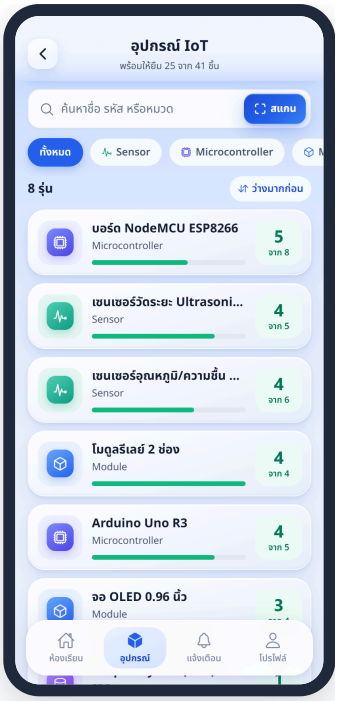
- **แถบเขียวด้านบน**: ตอนนี้มีผู้ดูแลอยู่ที่ห้องไหม (ผู้ดูแลกด "เช็กอิน" เมื่อมาถึง และระบบเช็กเอาท์ให้ทุกคนเองตอน 17:00)
- **ตัวเลข 3 ช่อง**: เครื่องคอมทั้งหมด / ใช้งานได้ / มีปัญหา
- **การ์ดห้อง**: กดเข้าไปดูผังห้องคอม (ส่วน 5)
- **ปุ่มฟ้า "สแกนยืม / คืนอุปกรณ์"**: ทางลัดไปหน้าสแกน
- **แถบล่าง** 4 แท็บ: ห้องเรียน · อุปกรณ์ · แจ้งเตือน · โปรไฟล์

พูด พอเปิดแอปมา นักศึกษาจะเห็นทันทีว่าตอนนี้มีผู้ดูแลอยู่ห้องไหม จะได้ไม่ต้องเดินไปเก้อ

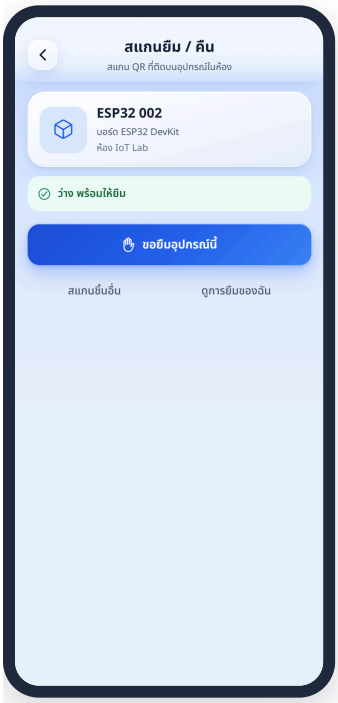
แท็บอุปกรณ์: รู้ว่าของว่างก่อนเดินไปห้อง

- รวมอุปกรณ์เป็น**รุ่น** เช่น NodeMCU มี 8 ชิ้น ว่าง 5 ชิ้น
- ค้นหาด้วยชื่อ รหัส หรือหมวด · กรองตามหมวด (Sensor, Microcontroller ฯลฯ)
- ถ้าของหมด แอปบอกว่า "ว่างประมาณวันไหน" จากวันครบกำหนดคืนของคนที่ยืมอยู่
- กดการ์ดเพื่อดูรายละเอียด และปุ่มเปิดกล้องสแกน หรือขอยืมแบบนัดรับ

> **พูด** นักศึกษาเช็กจากที่ไหนก็ได้ว่าของที่อยากยืมเหลือกี่ชิ้น ถ้าหมดก็รู้ว่าจะว่างประมาณวันไหน



แท็บอุปกรณ์



สแกนแล้วเจอชิ้นที่ว่าง

สแกน QR → ขอยืม

- ป้ายที่ติดอุปกรณ์มี **QR + รหัส 4 ตัว** ถ้าสแกนไม่ติดพิมพ์รหัส 4 ตัวแทนได้
- แอปบอกสถานะชิ้นนั้นทันที: ว่าง / ถูกยืมอยู่ (คืนวันไหน) / ของเราเอง
- กด **"ขอยืมอุปกรณ์นี้"** → เลือกจำนวนวัน (3 / 5 / 7 วัน) → เลือกสภาพ ปกติ หรือ ชำรุด (ชำรุดต้องเขียนว่าตรงไหน) → **ถ่ายรูปสดจากกล้อง** (เลือกรูปจากอัลบั้มไม่ได้)
- ก่อนส่ง แอปเด้ง**กฎการยืม-คืน**ให้อ่านและติ๊กยอมรับทุกครั้ง
- ส่งแล้ว ผู้ดูแลมีเวลาตอบ 30 นาที ไม่มีใครตอบ = คำขอหมดอายุ ส่งใหม่ได้

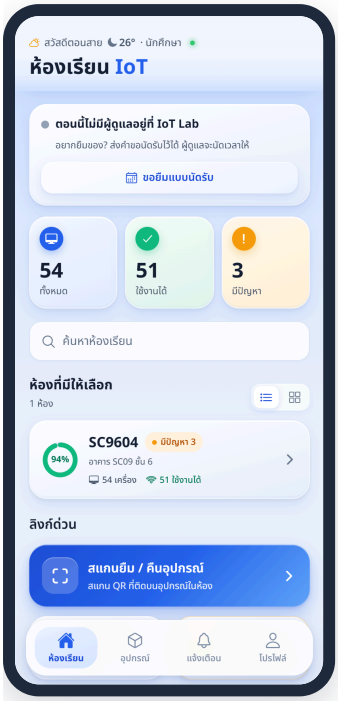
> **ทำไมต้องถ่ายรูปสด:** เป็นหลักฐานว่าตอนยืมของอยู่ในสภาพไหน ถ้าคืนมาแล้วเสีย จะรู้ว่าเสียตอนใคร รูปมีวันเวลาประทับไว้

กฎการยืมที่แอปบังคับให้เอง

3	3/5/7	30	1
ชิ้นสูงสุดต่อคน (รวมคำขอที่รอ + นัดรับ)	วัน ให้เลือกระยะยืม	นาที คำขอรอผู้ดูแลตอบ	ครั้ง ยืมต่อได้ (ต้องขอภายในวันครบกำหนด)

- **ขอคืนแล้วไม่มีใครตอบใน 30 นาที** = ระบบบันทึกคืนให้อัตโนมัติ (ผู้ดูแลตรวจสภาพย้อนหลังได้)

- **เกินกำหนดแล้ว ยืมต่อไม่ได้** ต้องนำมาคืนก่อน · คืนแล้วพบว่าชำรุด ผู้ดูแลใส่ค่าเสียหายได้
- ตัวเลขทุกตัวข้างบน Admin ปรับได้ที่หน้าตั้งค่าระบบ



ตอนไม่มีผู้ดูแลอยู่ห้อง

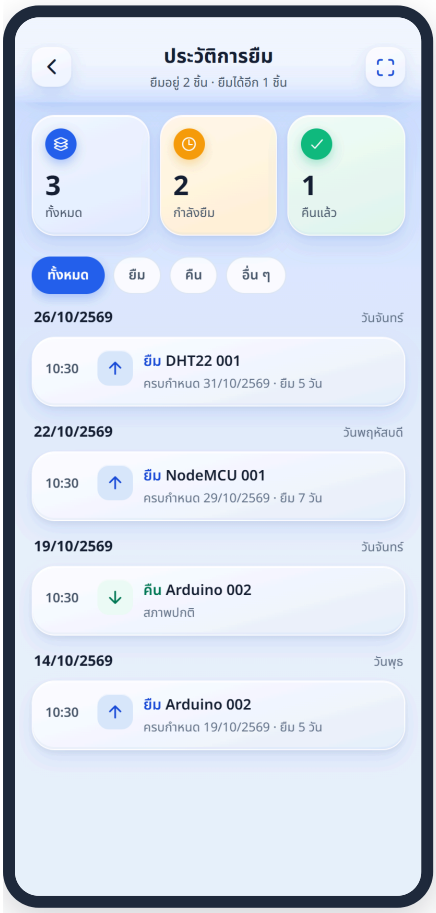
ขอยืมแบบนัดรับ (เมื่อไม่มีผู้ดูแลอยู่ห้อง)

การยืมต้องมีผู้ดูแลเปิดตู้ให้ ถ้าตอนนี้ไม่มีใครอยู่ นักศึกษา**ฝากคำขอนัดรับ**ไว้ได้:

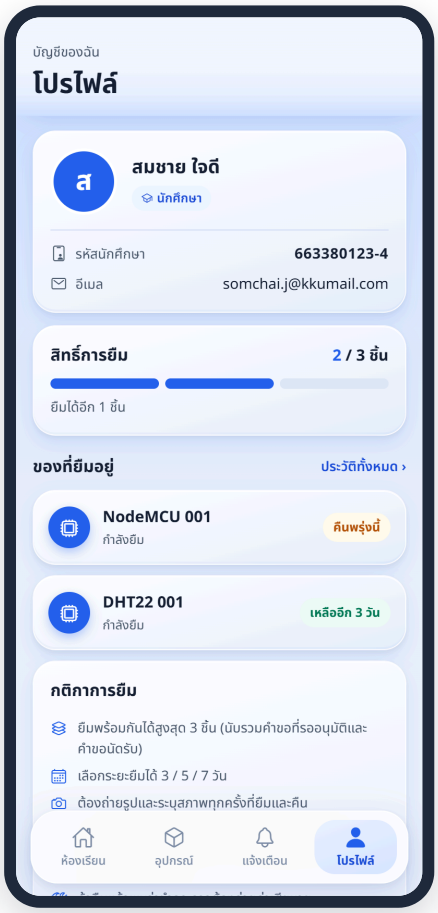
1. กด "ขอยืมแบบนัดรับ" เลือกรุ่น จำนวนวัน และเขียนหมายเหตุ เช่น "สะดวกช่วงบ่าย"
2. ผู้ดูแลกำหนดวัน-เวลานัด (หรือปฏิเสธพร้อมเหตุผล) นักศึกษาได้แจ้งเตือน
3. มารับตามนัด สแกน QR ชิ้นที่ได้ แล้วกด "ยืนยันรับของตามนัด"

- ระบบ**กันของไว้ให้ตั้งแต่ส่งคำขอ** คนอื่นจะยืมชิ้นสุดท้ายไปไม่ได้
- ผู้ดูแลไม่นัดภายใน 12 ชั่วโมง = คำขอหมดอายุ · มาช้าเกิน 10 นาที = นัดถูกยกเลิก (ไม่มาตามนัด)

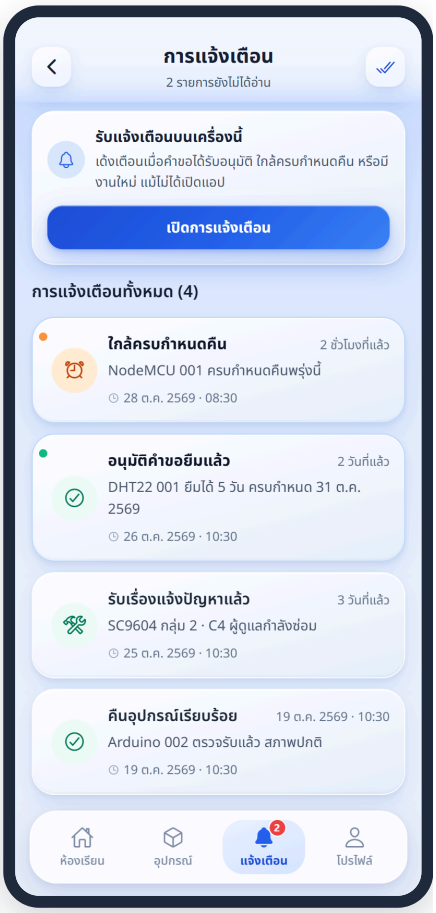
> **พูด** ฟีเจอร์นัดรับนี้ อาจารย์ที่ปรึกษาแนะนำเพิ่มเข้ามาครับ/ค่ะ แก้ปัญหานักศึกษามาแล้วไม่เจอผู้ดูแล



ประวัติการยืม เรียงตามวันแบบแอปธนาคาร



โปรไฟล์: สิทธิ์ยืม 2/3 + ของที่ยืมอยู่



แจ้งเตือน: ผลคำขอ + เตือนก่อนครบกำหนด

- **ประวัติการยืม**: ทุกรายการ ยืม / คืน / ยืมต่อ / นัดรับ เรียงตามวันเหมือนรายการเดินบัญชีในแอปธนาคาร ยกเลิกคำขอที่ยังรออยู่ได้จากหน้านี้
- **โปรไฟล์**: รหัสนักศึกษา (กรอกครั้งแรกครั้งเดียว แก้เองไม่ได้ ผิดต้องให้ผู้ดูแลแก้) · ยืมอยู่กี่ชิ้น · กติกาการยืม
- **แจ้งเตือน**: อนุมัติ/ปฏิเสธ · ทุกวัน 08:00 เตือนของที่ใกล้ครบกำหนด และของที่เกินกำหนด · เปิด "รับแจ้งเตือนบนเครื่องนี้" ได้ จะเด้งแม้ไม่ได้เปิดแอป

4 · งานของผู้ดูแล (TA / Admin)

เข้าแอปแล้วเจอแดชบอร์ดทันที · ใช้ได้ทั้งมือถือและคอม



แดชบอร์ดบนมือถือ

แดชบอร์ด

- ตัวเลขวันนี้: ยืมกี่ชิ้น คืนกี่ชิ้น งานซ่อมค้าง
- **การ์ดเช็กอิน**: มาถึงห้องกด "เช็กอิน" นักศึกษาจะเห็นว่ามีผู้ดูแลอยู่ กลับกด "เช็กเอาท์" (ลืมกด ระบบออกให้ 17:00)
- **กล่องคำขอ** พร้อมตัวเลขแดง = มีคำขอรออยู่กี่รายการ
- ทางลัด: สแกนดูสถานะ · เพิ่มอุปกรณ์ · จัดการอุปกรณ์ · ประวัติยืม · ห้องคอม · รายงาน · ตั้งค่า
- ผู้ดูแลอยากยืมของเอง กด "ยืมของ" สลับไปหน้านักศึกษา

กล่องคำขอ: อนุมัติในไม่กี่วินาที

- คำขอใหม่**ขึ้นเองทันที**ไม่ต้องรีเฟรช (อัปเดตสด) พร้อมนับถอยหลังเวลาที่เหลือ
- เห็นชื่อ + รหัสนักศึกษา, ชิ้นไหน, กี่วัน, สภาพที่นักศึกษาแจ้ง และ**รูปถ่าย** (กดขยายได้)
- คำขอคืน: เทียบรูปตอนยืมกับตอนคืนได้ ถ้าชำรุดใส่ค่าเสียหายได้
- แท็บ **นัดรับ**: เลือกเวลา "ตอนนี้ / อีก 10 นาที / อีก 30 นาที" หรือกำหนดเอง
- แท็บ **ประวัติ**: คำขอที่ตัดสินไปแล้ว ใครอนุมัติ เมื่อไร

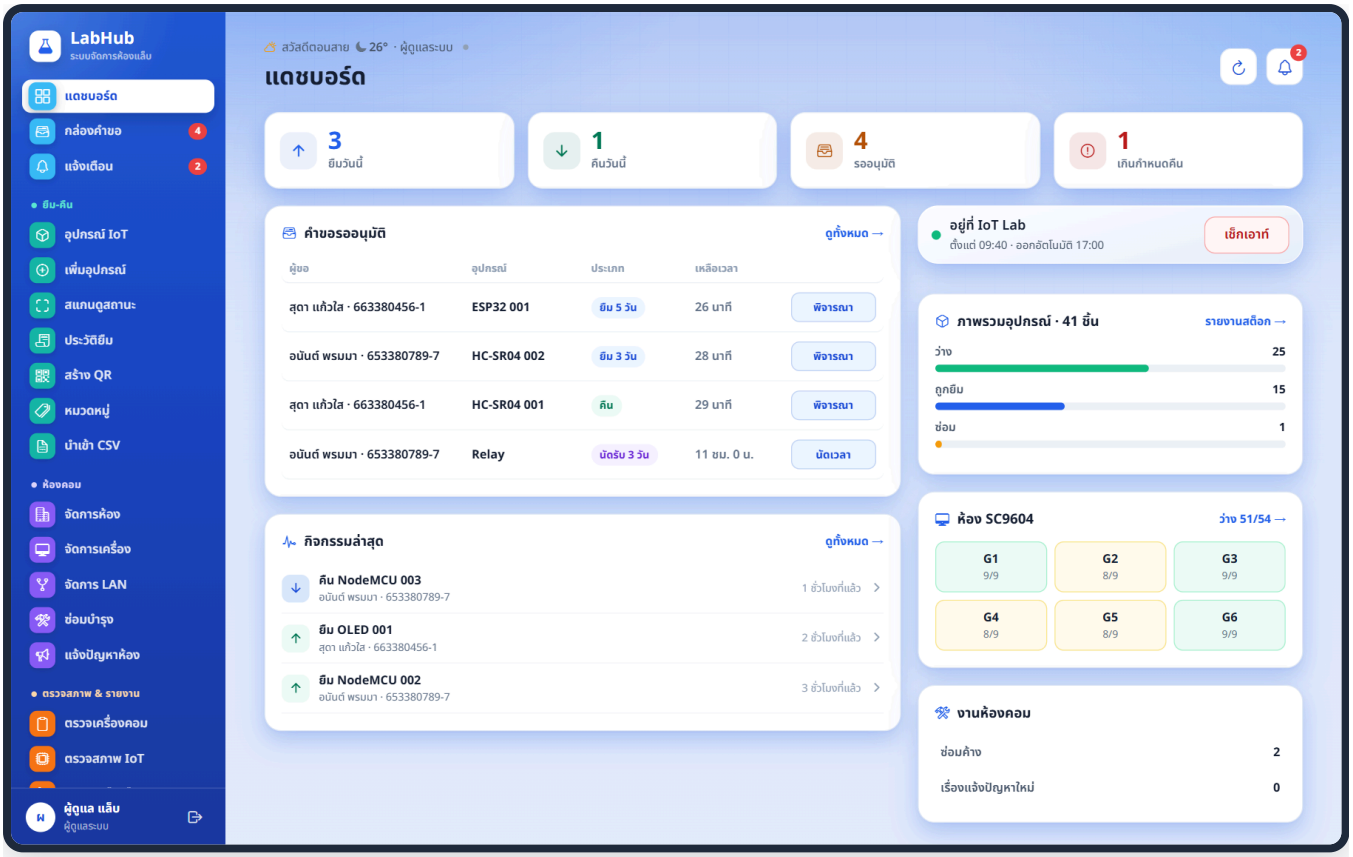
> **พูด** พอนักศึกษากดส่ง คำขอจะเด้งขึ้นมาที่เครื่องผู้ดูแลทันที ผู้ดูแลดูรูปแล้วกดอนุมัติได้เลย



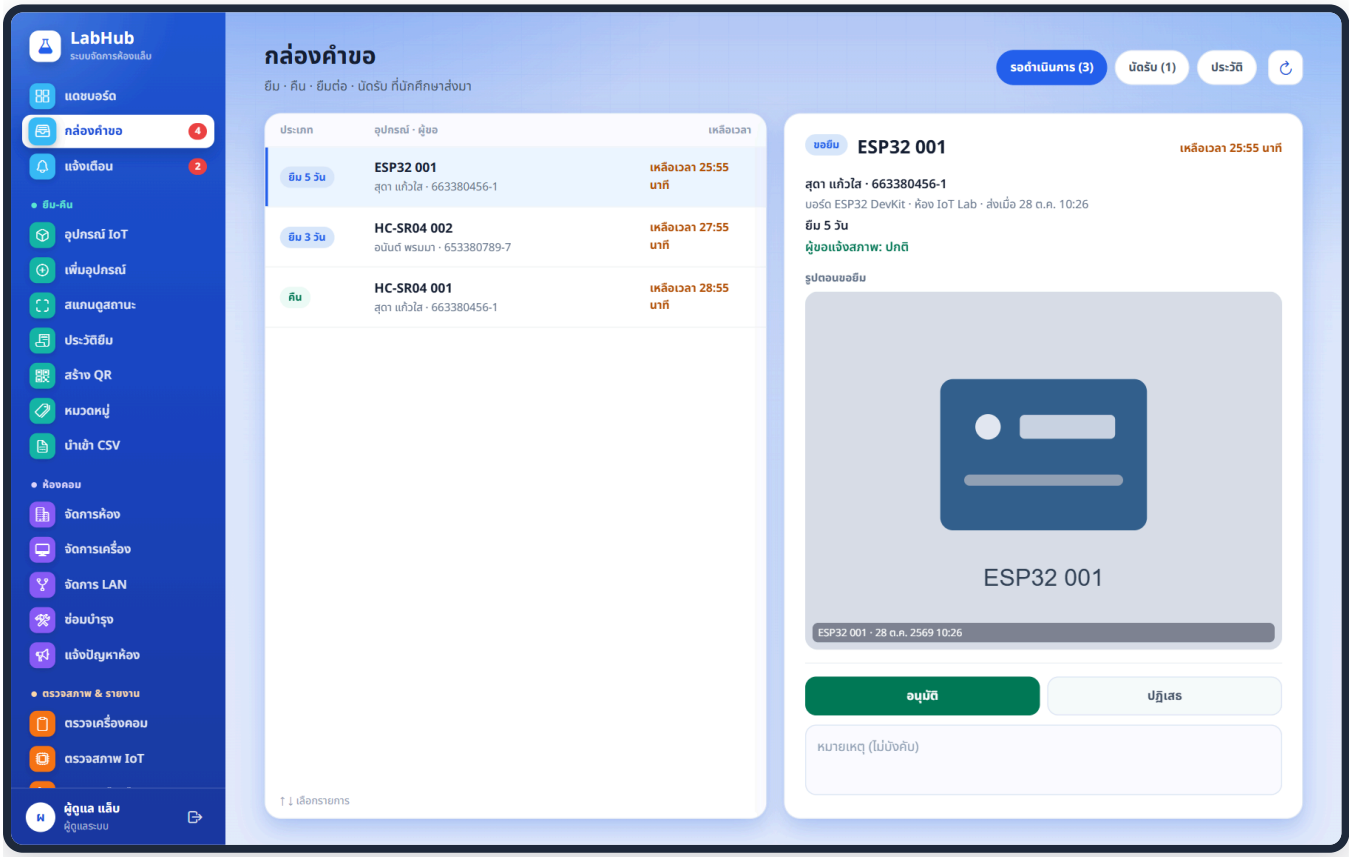
กล่องคำขอบนมือถือ

บนคอมจอใหญ่: หน้าตาแบบโปรแกรมสำนักงาน

เปิดเว็บบนคอม (จอกว้าง) หน้าผู้ดูแลจะเปลี่ยนเป็นแบบมี**เมนูซ้าย** ตาราง และหลายคอลัมน์ ทำงานได้ครบเหมือนมือถือทุกอย่าง แค่จัดหน้าใหม่ให้เหมาะกับจอใหญ่



แดชบอร์ดบนคอม: คำขอรออนุมัติ กิจกรรมล่าสุด ภาพรวมอุปกรณ์ ห้องคอม



กล่องคำขอบนคอม: รายการซ้าย รายละเอียด + รูปขวา

เครื่องมืออื่นของผู้ดูแล

เพิ่มอุปกรณ์
ถ่ายรูป + ชื่อ + หมวด ระบบ**ออกรหัสให้เอง** เช่น DHT22 012 พร้อมรหัสสแกน 4 ตัว

นำเข้า CSV
เพิ่มทีละหลายสิบชิ้นจากไฟล์ Excel ระบบตรวจก่อนนำเข้า

พิมพ์ป้าย QR
เลือกหลายชิ้น พิมพ์เป็นแผ่นสติกเกอร์ A4 ติดอุปกรณ์

สแกนดูสถานะ
สแกนชิ้นไหนก็ได้ รู้ว่าใครยืม คืนเมื่อไร ประวัติย้อนหลัง

รายงานยืม-คืน
เลือกช่วงเวลา ส่งออก Excel (CSV) / PDF · อุปกรณ์และผู้ยืมที่ยืมบ่อย

รายงานสต็อก
จำนวนตามสถานะ · **เตือนประกันใกล้หมด** · อุปกรณ์อายุมากที่ควรตรวจ/เปลี่ยน

ตรวจสภาพ IoT ประจำเทอม
ไล่ตรวจทีละชิ้น ใช้งานได้ / ซ่อม / เสีย

จัดการผู้ใช้
ค้นหา · แก้รหัสนักศึกษา · ตั้งสิทธิ์ user / TA / Admin

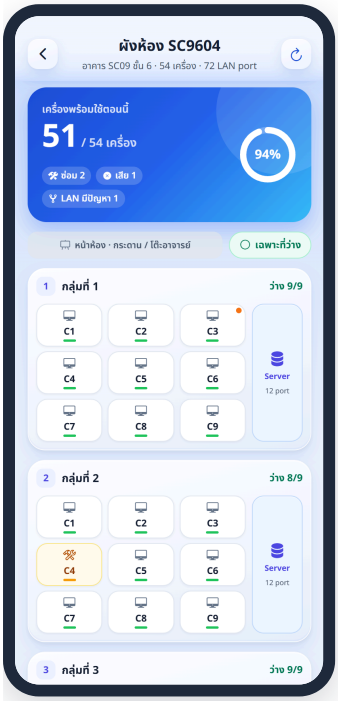
ตั้งค่าระบบ
จำนวนยืมสูงสุด · เวลาหมดอายุ · วันเตือนประกัน · กฎนัดรับ

สีม่วง = TA ใช้ได้ด้วย · สีส้ม = Admin เท่านั้น

5 · ระบบห้องคอมพิวเตอร์

แยกจากระบบยืม-คืนเด็ดขาด ทั้งข้อมูลและหน้าจอ

ห้องคอม 1 ห้อง แบ่งเป็น **6 กลุ่ม** แต่ละกลุ่มมี**คอม 9 เครื่อง (C1–C9)** และ**ตู้ Server ที่มีช่องสาย LAN 12 ช่อง** (รวม 54 เครื่อง 72 ช่อง LAN) Admin เพิ่มห้องใหม่ได้ในแอป



ผังห้อง SC9604

ผังห้อง: เห็นเครื่องว่างก่อนเข้าห้อง

- แถบฟ้าด้านบน: พร้อมใช้กี่เครื่อง คิดเป็นกี่เปอร์เซ็นต์ ซ่อมกี่เครื่อง เสียกี่เครื่อง
- สีของแต่ละเครื่อง: ขาว + ขีดเขียว = ใช้ได้ เหลือง = กำลังซ่อม แดง = เสีย
- จุดสีส้มมุมเครื่อง = เมาส์/คีย์บอร์ด/จอ มีปัญหา (เช่น เมาส์หาย)
- ปุ่ม **"เฉพาะที่ว่าง"** ซ่อนเครื่องที่ใช้ไม่ได้
- แตะเครื่อง: ดูอุปกรณ์ต่อพ่วง และ**"แจ้งปัญหาเครื่องนี้"** · แตะ Server: เลือกช่อง LAN ที่จะแจ้ง

> **พูด** นักศึกษาเปิดดูผังห้องก่อนได้เลยว่าเครื่องไหนว่าง เครื่องไหนเสีย ถ้าเจอเครื่องมีปัญหา แตะแล้วแจ้งได้ในไม่กี่วินาที

เส้นทางเมื่อนักศึกษาแจ้งปัญหา

นักศึกษาแจ้ง
เช่น "จอไม่ติด"

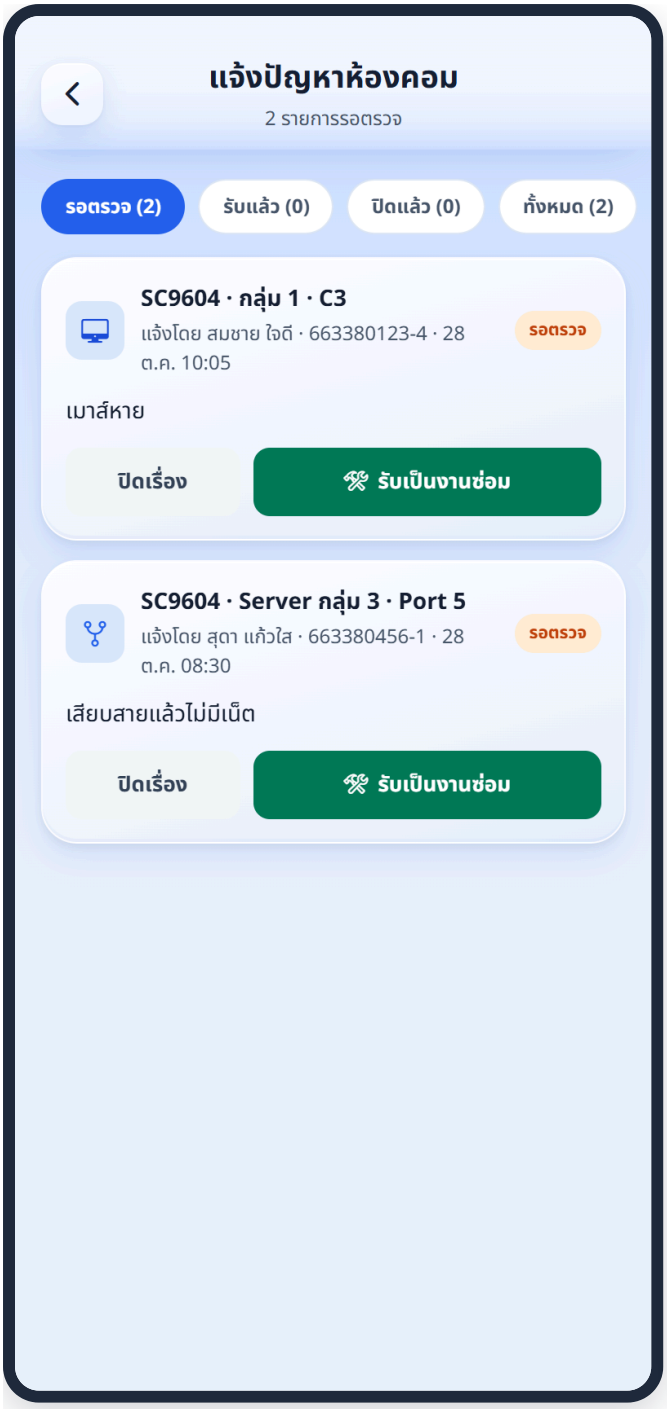
→ **เข้าคิวผู้ดูแล**
หน้าแจ้งปัญหาห้องคอม

→ **รับเป็นงานซ่อม**
เครื่องเปลี่ยนเป็น "กำลังซ่อม"

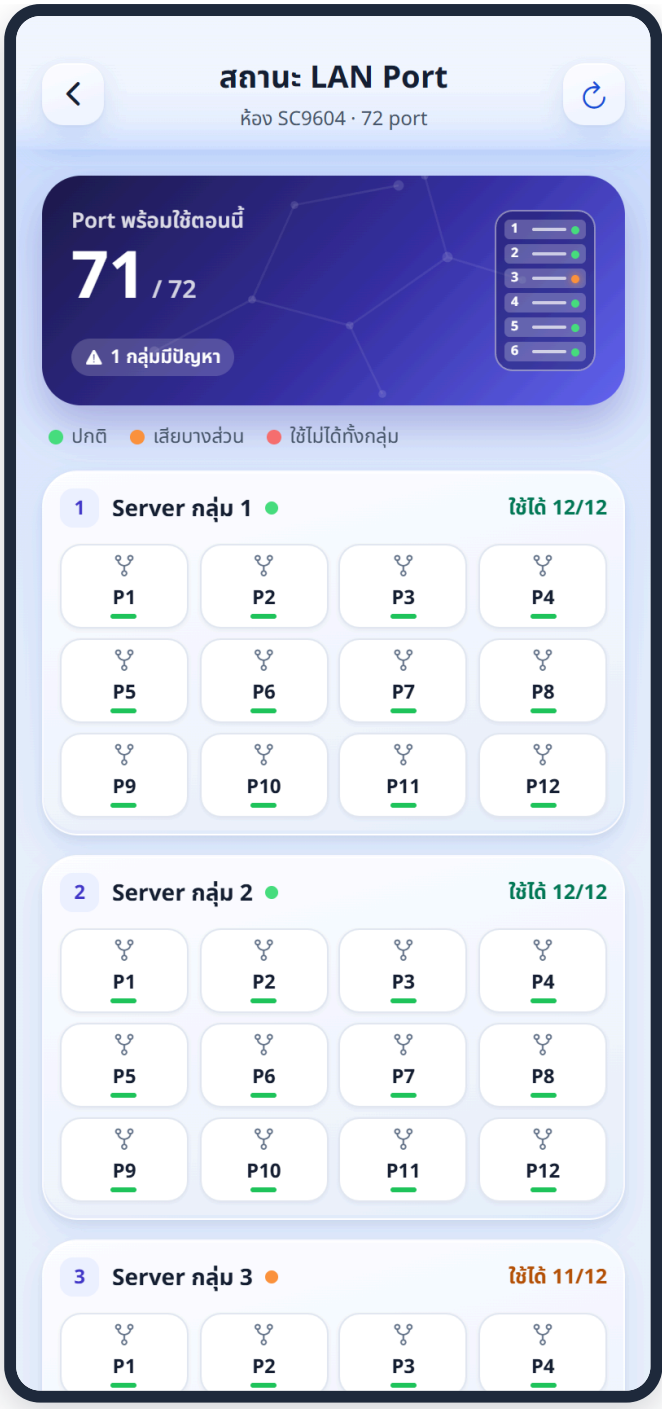
→ **ซ่อม**
รอซ่อม → กำลังซ่อม → เสร็จ

→ **เครื่องกลับมาใช้ได้**
สถานะเปลี่ยนเอง

- ผู้แจ้งได้รับแจ้งเตือนผลตอนผู้ดูแลรับเรื่องหรือปิดเรื่อง
- ถ้าเครื่องไม่ได้เสียจริง ผู้ดูแลกด "ปิดเรื่อง" พร้อมเหตุผล (ผู้แจ้งเห็นเหตุผล)
- **กันแจ้งมั่ว:** เรื่องเดิมที่ยังเปิดอยู่แจ้งซ้ำไม่ได้ · คนละไม่เกิน 5 ครั้งต่อวัน
- ทุกครั้งที่สถานะเครื่องเปลี่ยน ระบบ**บันทึกประวัติเอง** ว่าใครเปลี่ยน เมื่อไร



คิวแจ้งปัญหาห้องคอม (ฝั่งผู้ดูแล)



สถานะสาย LAN ทุกตู้ Server

งานอื่นในระบบห้องคอม

เช็กลิสต์อุปกรณ์

เมาส์ / คีย์บอร์ด / จอ ของแต่ละเครื่อง: ครบ · หาย · ชำรุด

ตรวจประจำเทอม

ไล่ตรวจทุกเครื่อง ผลตรวจอัปเดตเช็กลิสต์ให้เอง

จัดการห้อง/เครื่อง/LAN

Admin เพิ่ม แก้ ปิดใช้งาน · TA เปลี่ยนสถานะได้

6 ใช้ได้บนอะไรบ้าง



หน้าเข้าสู่ระบบ

แอปมือถือ
iPhone และ Android จากโค้ดชุดเดียวกัน

เว็บไซต์
labhub-kku.vercel.app เปิดได้ทุกเบราว์เซอร์

ติดตั้งเว็บเป็นแอป
กด "ติดตั้ง" ได้ไอคอนบนหน้าจอโฮม เปิดเต็มจอเหมือนแอป (PWA)

แจ้งเตือนเด้ง
เว็บที่ติดตั้งแล้วรับแจ้งเตือนได้ แม้ไม่ได้เปิดอยู่

ข้อจำกัดที่ควรรู้: การ**ขอยืม/คืน**ต้องทำบน**มือถือ** (แอปหรือเว็บบนมือถือ) เพราะต้องถ่ายรูปสดจากกล้อง บนคอมจะดูได้อย่างเดียว · เข้าระบบได้เฉพาะบัญชี Google ของ มข. (นักศึกษา/บุคลากร) ส่วนรหัสผ่านใช้สำหรับบัญชีผู้ดูแล · ไม่มีหน้าสมัครสมาชิก/ลืมรหัสผ่าน (จำรหัสไม่ได้ ติดต่อผู้ดูแล)

7 พัฒนายังไง ใช้อะไร

พูด แอปนี้แบ่งเป็น 2 ส่วน คือ**หน้าบ้าน**ที่ผู้ใช้เห็นและกด กับ**หลังบ้าน**ที่เก็บข้อมูลและตัดสินกติกา หน้าบ้านเราเขียนครั้งเดียวแล้วได้ทั้งแอป iPhone, Android และเว็บ ส่วนหลังบ้านใช้บริการชื่อ Supabase ที่มีฐานข้อมูล ระบบล็อกอิน และที่เก็บรูปพร้อมในตัว

ภาพรวมระบบ

ผู้ใช้
มือถือ / เว็บ / คอม

→

หน้าบ้าน (แอป)
React Native + Expo
เขียนด้วย TypeScript

⇄

หลังบ้าน Supabase
ฐานข้อมูล · ล็อกอิน · เก็บรูป · อัปเดตสด · งานตามเวลา

→

บริการภายนอก
Google (ล็อกอิน มข.)
Vercel (ฝากเว็บ)
Web Push (เด้งแจ้งเตือน)

เครื่องมือที่ใช้ อธิบายแบบไม่ต้องรู้โค้ด

เครื่องมือ	ใช้ทำอะไร
React Native + Expo	ชุดเครื่องมือสร้างแอป เขียนโค้ดชุดเดียวได้ทั้ง iPhone, Android และเว็บ ไม่ต้องเขียน 3 รอบ เหมือนแม่พิมพ์เดียว ปั๊มขนมได้ 3 แบบ
TypeScript	ภาษาที่ใช้เขียนแอป (JavaScript ที่ตรวจความถูกต้องเข้มขึ้น) ช่วยจับข้อผิดพลาดตั้งแต่ตอนเขียน
Expo Router	ระบบเปลี่ยนหน้าในแอป แต่ละหน้าคือไฟล์หนึ่งไฟล์
Supabase	หลังบ้านสำเร็จรูป มีครบในที่เดียว: · **ฐานข้อมูล PostgreSQL** เก็บข้อมูลเป็นตาราง (23 ตาราง เช่น อุปกรณ์ การยืม ผู้ใช้ ห้อง เครื่องคอม) · **Auth** ระบบล็อกอิน (Google มข.) · **Storage** เก็บรูปอุปกรณ์และรูปหลักฐานยืม-คืน · **Realtime** ส่งสัญญาณให้หน้าจออัปเดตเองทันที · **งานตามเวลา (cron)** ทำงานอัตโนมัติ เช่น ตัดคำขอหมดอายุ เตือน 08:00 เช็กเอาท์ 17:00 · **Edge Function** โปรแกรมเล็ก ๆ บนเซิร์ฟเวอร์ ใช้ส่งแจ้งเตือนเด้ง เหมือนเช่าออฟฟิศที่มีตู้เก็บเอกสาร รปภ. และพนักงานส่งของพร้อม
Google Sign-In	เข้าระบบด้วยบัญชีมหาวิทยาลัย ไม่ต้องเก็บรหัสผ่านนักศึกษาเอง
Vercel	ที่ฝากเว็บไซต์ (labhub-kku.vercel.app) แก้โค้ดแล้วเว็บอัปเดตเอง
PWA + Web Push	ทำให้เว็บติดตั้งเป็นไอคอนบนมือถือได้ และเด้งแจ้งเตือนได้
GitHub	ที่เก็บโค้ดออนไลน์ ทุกการแก้ไขมีประวัติย้อนดูได้ (มากกว่า 190 ครั้ง) ทำงานร่วมกันผ่าน Pull Request

จุดเด่นด้านการออกแบบระบบ (ใช้ตอบกรรมการ)

กติกาอยู่ที่หลังบ้าน ไม่ใช่ในแอป

แอปแค่ "ส่งคำขอ" ฐานข้อมูลเป็นคนตัดสินว่ายืมได้ไหม เกินโควตาไหม ต่อให้มีคนแก้แอปก็โกงกติกาไม่ได้

เหมือนเคาน์เตอร์ธนาคาร: ลูกค้ากรอกใบถอน แต่พนักงานเป็นคนเช็กยอด

ตรวจสิทธิ์ทุกแถวข้อมูล (RLS)

นักศึกษาเห็นแค่ประวัติของตัวเอง ยังไม่ล็อกอินเข้าอะไรไม่ได้ ตารางใหม่ถูกเปิดระบบตรวจสิทธิ์ให้อัตโนมัติ

เหมือน รปภ. ประจำทุกลิ้นชัก

หลักฐานรูปถ่ายทุกครั้ง
ถ่ายสดเท่านั้น ย่อขนาดก่อนอัปโหลด ประหยัดพื้นที่และเน็ต

อัปเดตสด
คำขอใหม่ เช็กอินผู้ดูแล สถานะเครื่องคอม ขึ้นหน้าจอทันทีโดยไม่ต้องกดรีเฟรช

ทดสอบอัตโนมัติ 53 ข้อ
ชุดทดสอบกติกายืม-คืน (ยืม อนุมัติ คืน ยืมต่อ หมดเวลา โควตา นัดรับ) รันได้โดยไม่แตะข้อมูลจริง

ใช้แพ็กเกจฟรีได้
ออกแบบให้ยิงฐานข้อมูลน้อย (เช่น งานอัตโนมัติไม่ถี่เกิน) จึงไม่มีค่าใช้จ่ายรายเดือน

เส้นทางการพัฒนา

ช่วงเวลา (2569)	สิ่งที่ทำ
มีนาคม	เริ่มโปรเจกต์ (17 มี.ค.) · ทดลองใช้ฐานข้อมูล MongoDB แล้วเปลี่ยนมาใช้ Supabase เพราะมีระบบล็อกอิน เก็บรูป และอัปเดตสดในตัว · หน้าจัดการอุปกรณ์
พฤษภาคม	ระบบสแกน QR · ออกแบบหน้าตาใหม่ทั้งแอป (ธีมฟ้า)
ต้นตุลาคม	ยืม-คืนเฟสแรกเสร็จ (2 ต.ค.) · บทบาท TA · ระบบห้องคอมครบ: ผังห้อง ประวัติสถานะ เช็กลิสต์ แจ้งปัญหา · รหัสนักศึกษา
10–11 ต.ค.	ฟีเจอร์ตามคำแนะนำอาจารย์ F1–F4: ป๊อปอัปกฎการยืม · ประวัติแบบแอปธนาคาร · เช็กอินผู้ดูแล · ขอยืมแบบนัดรับ
12–19 ต.ค.	หน้าเข้าสู่ระบบใหม่ (Google มข.) · เว็บติดตั้งเป็นแอป + แจ้งเตือนเด้ง · หน้าผู้ดูแลแบบจอใหญ่ · ชุดทดสอบ 53 ข้อ · ระบบสำรองข้อมูล
22–28 ต.ค.	รองรับบุคลากร มข. (@kku.ac.th) · ปรับความปลอดภัยเพิ่ม · จัดระเบียบโค้ดทั้งโปรเจกต์ · ทำแผนภาพประกอบรายงาน

ตัวเลขน่าสนใจ

2
ระบบในแอปเดียว

30+
หน้าจอ

23
ตารางในฐานข้อมูล

190+
ครั้งที่บันทึกการแก้ไขโค้ด

8 · บทเดโมสด

ซ้อมอย่างน้อย 2 รอบกับเครื่องจริงก่อนวันนำเสนอ

เตรียมก่อนวันนำเสนอ

	รายการ
☐	**มือถือ 1 เครื่อง** ล็อกอินบัญชี**นักศึกษาทดสอบ** (กรอกรหัสนักศึกษาแล้ว) ติดตั้งแอปหรือเว็บไว้บนหน้าจอโฮม และอนุญาตกล้องแล้ว
☐	**โน้ตบุ๊ก** เปิด labhub-kku.vercel.app ล็อกอินบัญชี **TA หรือ Admin** (คนละบัญชีกับมือถือ เพราะอนุมัติคำขอของตัวเองไม่ได้) ต่อจอโปรเจกเตอร์
☐	**อุปกรณ์ทดสอบ 1 ชิ้นที่ติดป้าย QR** สถานะ "ว่าง" (จดรหัส 4 ตัวบนป้ายไว้ เผื่อสแกนไม่ติด)
☐	บัญชีนักศึกษาทดสอบ**ยืมค้างไม่ถึง 3 ชิ้น** (ไม่งั้นปุ่มขอยืมกดไม่ได้)
☐	**เปิดแอปเล่นสักครั้งในวันก่อนนำเสนอ** (หลังบ้านแพ็กเกจฟรีจะหลับถ้าไม่มีคนใช้นานราว 7 วัน)
☐	เน็ตมือถือสำรอง (เปิดฮอตสปอต) เผื่อ Wi-Fi ห้องนำเสนอมีปัญหา
☐	**แผนสำรอง:** อัดวิดีโอหน้าจอเดโมเก็บไว้ 1 รอบ + ใช้ภาพในคู่มือนี้

ลำดับเดโม (ประมาณ 11 นาที)

#	ทำอะไร	พูดอะไร
1	คนที่ 1 โชว์หน้าแรกบนมือถือ (ต่อจอหรือถือให้กล้องจับ)	"นี่คือหน้าแรกของนักศึกษา เห็นว่าตอนนี้มีผู้ดูแลอยู่ห้องไหม และสถานะห้องคอม"
2	คนที่ 2 กด "เช็กอิน" บนแดชบอร์ดโน้ตบุ๊ก	"ผู้ดูแลมาถึงห้องกดเช็กอิน" → **ให้ดูมือถือ** แถบเปลี่ยนเป็นสีเขียวเองทันที ถ้าปุ่มขึ้นว่า "เช็กเอาท์" แปลว่าเช็กอินค้างไว้ ให้กดเช็กเอาท์ก่อนเริ่มนำเสนอ
3	คนที่ 1 แท็บอุปกรณ์ → ค้นหาชื่ออุปกรณ์ทดสอบ	"เช็กได้ก่อนว่าของที่อยากยืมเหลือกี่ชิ้น"
4	กดสแกน → สแกน QR บนอุปกรณ์ทดสอบ	"ทุกชิ้นมีป้าย QR สแกนแล้วรู้ทันทีว่าชิ้นนี้ว่าง"
5	ขอยืมอุปกรณ์นี้ → 3 วัน → สภาพปกติ → ถ่ายรูป → ติ๊กยอมรับกฎ → ส่ง	"ต้องถ่ายรูปสดทุกครั้งเป็นหลักฐาน และต้องยอมรับกฎก่อนส่ง"

#	ทำอะไร	พูดอะไร
6	คนที่ 2 ชี้ที่โน้ตบุ๊ก คำขอเด้งขึ้นในกล่องคำขอ → เปิดดูรูป → อนุมัติ	"คำขอขึ้นมาเองไม่ต้องรีเฟรช ผู้ดูแลเห็นรูปและชื่อนักศึกษา กดอนุมัติ"
7	คนที่ 1 เปิดแจ้งเตือน → ประวัติการยืม	"นักศึกษาได้แจ้งเตือนทันที และเห็นในประวัติ พร้อมวันครบกำหนดคืน"
8	สแกนชิ้นเดิมอีกครั้ง → ขอคืน → ถ่ายรูป → ส่ง · คนที่ 2 รับคืน	"ตอนคืนก็ถ่ายรูปเหมือนกัน ผู้ดูแลเทียบรูปก่อน-หลังได้"
9	คนที่ 2 เปิดผังห้องคอม → ชี้สีเครื่อง	"ระบบที่สอง ห้องคอม เห็นทุกเครื่องว่าใช้ได้หรือเสีย"
10	คนที่ 1 แตะเครื่องหนึ่งบนมือถือ → แจ้งปัญหา "ทดสอบระบบ" · คนที่ 2 เปิดคิวแจ้งปัญหา → ปิดเรื่องพร้อมเหตุผล "ทดสอบระหว่างนำเสนอ"	"นักศึกษาแจ้งปัญหาได้ทันที เรื่องเข้าคิวผู้ดูแล รับเป็นงานซ่อมหรือปิดเรื่องพร้อมเหตุผลได้"
11	คนที่ 2 (ถ้ามีเวลา) รายงานยืม-คืน → ส่งออก PDF	"ผู้ดูแลออกรายงานส่งอาจารย์ได้ในคลิกเดียว"

ข้อมูลที่เกิดจากเดโมเป็นข้อมูลจริงในระบบ ใช้อุปกรณ์และบัญชีทดสอบเท่านั้น ข้อ 10 ให้ "ปิดเรื่อง" (อย่ารับเป็นงานซ่อม ไม่งั้นเครื่องจริงจะขึ้นว่ากำลังซ่อม) หลังนำเสนอแจ้งพี่ให้ตรวจ/ลบข้อมูลทดสอบ

ถ้าเกิดปัญหาระหว่างเดโม

สแกน QR ไม่ติด
พิมพ์รหัส 4 ตัวใต้ QR ในช่อง "สแกนไม่ติด?" แทน

ปุ่มขอยืมกดไม่ได้
บัญชียืมครบ 3 ชิ้นแล้ว หรือชิ้นนั้นมีคนนัดรับไว้

คำขอไม่เด้งที่โน้ตบุ๊ก
กดปุ่มรีเฟรชมุมขวาบนของกล่องคำขอ

เน็ตล่ม / แอปโหลดไม่ขึ้น
สลับไปฮอตสปอต ถ้ายังไม่ได้ เปิดวิดีโอสำรองแล้วเล่าตามภาพ

9 คำถามที่กรรมการน่าจะถาม

เรื่องการใช้งาน

ถาม ถ้านักศึกษายืมแล้วไม่คืนล่ะ?

ตอบ ระบบเตือนทุกวัน 08:00 ก่อนครบกำหนดและเมื่อเกินกำหนด ผู้ดูแลก็ได้แจ้งเตือนด้วย เกินกำหนดแล้วยืมต่อไม่ได้ และยืมได้สูงสุดแค่ 3 ชิ้น ของที่ค้างนับรวมในโควตา จึงยืมชิ้นใหม่ไม่ได้จนกว่าจะคืน

ถาม ถ้าของคืนมาเสีย รู้ได้ยังไงว่าใครทำ?

ตอบ ทุกครั้งที่ยืมและคืนต้องถ่ายรูปสดและระบุสภาพ ผู้ดูแลเปิดรูปตอนยืมเทียบกับตอนคืนได้ ถ้าชำรุดผู้ดูแลบันทึกค่าเสียหายได้ตอนรับคืน

ถาม นักศึกษาสแกนแล้วหยิบของไปเองเลยได้ไหม?

ตอบ ไม่ได้ การสแกนเป็นแค่การส่งคำขอ ต้องมีผู้ดูแลอนุมัติ และของอยู่ในตู้ที่ผู้ดูแลเปิดให้ ถ้าไม่มีผู้ดูแลอยู่ห้องจะใช้ระบบนัดรับแทน

ถาม ถ้าผู้ดูแลไม่ว่างตอบคำขอ?

ตอบ คำขอยืมหมดอายุใน 30 นาที นักศึกษาส่งใหม่ได้ ส่วนคำขอคืนถ้าไม่มีใครตอบ ระบบบันทึกคืนให้อัตโนมัติ และผู้ดูแลตรวจสภาพย้อนหลังได้

ถาม ทำไมยืมบนคอมไม่ได้?

ตอบ ตั้งใจให้เป็นแบบนั้น เพราะต้องถ่ายรูปสดจากกล้องเป็นหลักฐาน และต้องอยู่หน้าของจริงตอนสแกน บนคอมจึงดูข้อมูลได้อย่างเดียว

ถาม ทำไมไม่ใช้ Google Sheet หรือ Excel?

ตอบ Sheet จดข้อมูลได้ แต่บังคับกติกาไม่ได้ เช่น โควตา 3 ชิ้น หมดเวลา 30 นาที การกันของให้คนนัดรับ ไม่มีรูปหลักฐาน ไม่มีแจ้งเตือน และใครมีสิทธิ์แก้ก็แก้ได้ทุกช่อง

เรื่องเทคนิคและความปลอดภัย

ถาม ข้อมูลปลอดภัยไหม? นักศึกษาเห็นข้อมูลคนอื่นได้ไหม?

ตอบ ฐานข้อมูลตรวจสิทธิ์ทุกครั้งที่อ่านหรือเขียน (Row Level Security) นักศึกษาเห็นแค่ประวัติของตัวเอง ยังไม่ล็อกอินเข้าอะไรไม่ได้ เว็บเข้ารหัส SSL และกติกาสำคัญทั้งหมดตัดสินที่ฐานข้อมูล ไม่ใช่ที่แอป จึงแก้แอปเพื่อโกงไม่ได้

ถาม ถ้ามีคนแก้แอปให้ยืมเกิน 3 ชิ้นได้ล่ะ?

ตอบ ทำไม่ได้ เพราะแอปแค่ "ขอ" ฐานข้อมูลเป็นคนนับและตัดสินเอง ถ้าเกินโควตาฐานข้อมูลก็ปฏิเสธ

ถาม ทำไมเลือก Supabase ไม่ทำเซิร์ฟเวอร์เอง?

ตอบ ได้ฐานข้อมูล ระบบล็อกอิน ที่เก็บรูป และการอัปเดตสด ครบในที่เดียว ไม่ต้องดูแลเครื่องเซิร์ฟเวอร์เอง และใช้แพ็กเกจฟรีได้ ตอนแรกเคยลอง MongoDB แล้วเปลี่ยนมาใช้ตัวนี้

ถาม ทำไมใช้ React Native?

ตอบ เขียนครั้งเดียวได้ทั้ง iPhone, Android และเว็บ ทีมเล็กดูแลโค้ดชุดเดียวได้

ถาม มีค่าใช้จ่ายไหม?

ตอบ ตอนนี้ใช้แพ็กเกจฟรีทั้งหมด (Supabase และ Vercel) ถ้าผู้ใช้เยอะขึ้นหรืออยากได้ระบบสำรองข้อมูลอัตโนมัติ ค่อยขยับเป็นแพ็กเกจเสียเงิน

ถาม ทดสอบระบบยังไง?

ตอบ มีชุดทดสอบอัตโนมัติ 53 ข้อสำหรับกติกายืม-คืน (ยืม อนุมัติ ปฏิเสธ ยืมต่อ คืนปกติ คืนชำรุด หมดเวลา โควตานัดรับ) รันแล้วย้อนข้อมูลทดสอบกลับทั้งหมด ไม่กระทบข้อมูลจริง และลองใช้บนมือถือจริงด้วย

ถาม ข้อมูลหายจะทำยังไง?

ตอบ มีสคริปต์สำรองข้อมูลทุกตารางเป็นไฟล์ (เปิดด้วย Excel ได้) แนะนำให้สำรองเดือนละครั้ง และมีคู่มือการกู้คืน

ถาม ใช้ AI ช่วยเขียนไหม?

ตอบ ตกลงกับพี่ก่อนว่าจะตอบแบบนี้ ใช้ AI ผู้ช่วยเขียนโค้ด (Claude Code) ช่วยในหลายส่วน แต่ทีมเป็นคนกำหนดฟีเจอร์ ออกแบบหน้าจอ (ทำแบบร่างให้ตรวจก่อนทุกครั้ง) ทดสอบบนมือถือจริง และตรวจผลทุกครั้ง

เรื่องอนาคต

ถาม จะพัฒนาต่อยังไง?

ตอบ เปิดสวิตช์รับเฉพาะอีเมล มข. เมื่อใช้จริง · ขยายไปห้องแล็บอื่นได้เพราะเพิ่มห้องในแอปได้อยู่แล้ว · ระบบสำรองข้อมูลอัตโนมัติ · อาจเพิ่มการจองเครื่องคอมล่วงหน้า เลือก/ปรับตามแผนจริงของทีม

10 · อ่านโค้ดสำหรับคนไม่เขียนโค้ด

เผื่ออาจารย์เปิดโค้ดแล้วถามว่า "ตรงนี้คืออะไร ทำงานยังไง"

ไม่ต้องเขียนโค้ดเป็น แค่**รู้ว่าโค้ดอยู่ตรงไหน** และ**อ่านออกทีละบรรทัด**ก็พอ ส่วนนี้เรียงจากง่ายไปยาก:

1. **หน้านี้** แผนที่โฟลเดอร์ + คอมเมนต์คืออะไร
2. **10.1 ปูพื้นภาษาโค้ด** หน้าตาโค้ด 12 แบบ + method 8 ตัว + วิธีแยกบรรทัดทีละคำ
3. **10.2 อ่านโค้ดจริง 3 รูป** จาก app/scan.tsx ไม่มีคอมเมนต์ มีเลขบรรทัด
4. **10.3 ท่อนเสริม** กติกาหลังบ้านและไฟล์ตัวช่วย (อ่านเพิ่มถ้ามีเวลา)

โค้ดอยู่ตรงไหน (แผนที่โฟลเดอร์)

โฟลเดอร์	เก็บอะไร
`app/`	**หน้าจอ** 1 ไฟล์ = 1 หน้า ชื่อไฟล์ = ที่อยู่หน้า เช่น `app/scan.tsx` = หน้าสแกน · `app/admin/` = หน้าของผู้ดูแล
`components/`	**ชิ้นส่วนหน้าจอ**ที่ใช้ซ้ำ เช่น ปุ่ม การ์ด แถบล่าง (`borrow/` = ของระบบยืม · `room/` = ของระบบห้องคอม)
`lib/`	**ตัวช่วย** เช่น ชื่อ/สีสถานะ สิทธิ์ของแต่ละบทบาท
`supabase/migrations/`	**ฐานข้อมูล + กติกา** เขียนด้วยภาษา SQL (ตาราง กฎสิทธิ์ และฟังก์ชันที่ตัดสินว่ายืมได้ไหม)

เคล็ดลับอ่านโค้ดแบบไม่ต้องเขียนเป็น: บรรทัดที่ขึ้นต้นด้วย `//` (ฝั่งแอป) หรือ `--` (ฝั่ง SQL) คือ**คอมเมนต์** โค้ดโปรเจกต์นี้เขียนคอมเมนต์เป็นภาษาไทยเกือบทุกไฟล์ อ่านตรงนั้นก่อนจะรู้ว่าท่อนนั้นทำอะไร · `if` = ถ้า · `return` = ส่งคำตอบกลับ · `raise exception` = ปฏิเสธพร้อมข้อความ

10.1 ปูพื้นภาษาโค้ด (อ่านก่อนดู 3 รูป)

โค้ด 1 บรรทัด ≈ **1 ประโยคคำสั่ง** อ่านจากซ้ายไปขวา เครื่องหมาย `;` ท้ายบรรทัด = จบประโยค

หน้านี้สอนเฉพาะ**รูปแบบ (syntax)** และ **method** ที่โผล่ใน 3 รูปถัดไป ทุกตัวมีเลขบรรทัดให้ชี้ตามได้ ไม่ต้องจำหมด เปิดหน้านี้ค้างไว้ข้าง ๆ ตอนอ่านโค้ดก็ได้

รูปแบบ (syntax) 12 แบบ ที่เจอใน 3 รูป

หน้าตา	อ่านว่า	ตัวอย่างจริง	เทียบกับ
`const ชื่อ = ค่า`	ตั้งชื่อให้ค่านี้ (เรียกว่า**ตัวแปร**)	บรรทัด 362 `const max = …`	แปะป้ายชื่อบนกล่อง
`"ข้อความ"`	ข้อความตรง ๆ	บรรทัด 380 `"borrow"`	คำในเครื่องหมายคำพูด
`` `…${ค่า}…` ``	ข้อความที่**เติมค่า**ลงไป	บรรทัด 206 `` `ส่งคำ${…} ${item.item_code} แล้ว` ``	แบบฟอร์มที่เว้นช่องให้เติม
`ก.ข`	"**ข ของ ก**" (จุด = ของ)	บรรทัด 362 `lookup.max_active_borrows` = เพดานยืม**ของ**ผลสแกน	"ชื่อ ของ นักศึกษา"
`{ ชื่อ: ค่า, … }`	กล่องข้อมูลหลายช่อง (**object**)	บรรทัด 195 `{ p_item_id: item.id, p_days: days, … }`	ใบฟอร์มที่มีหลายช่อง
`ก[ข]`	เปิดลิ้นชัก ก ที่ป้าย ข	บรรทัด 206 `KIND_TH[mode]` = คำไทยของโหมดนี้ เช่น borrow → "ขอยืม"	พจนานุกรม เปิดหาคำ
`ชื่อ(…)`	**สั่งให้ทำงาน** ของในวงเล็บ = ของที่ยื่นให้	บรรทัด 192 `uploadBorrowPhoto(photoUri)` = สั่งอัปโหลด ยื่นรูปให้	สั่งงานพร้อมยื่นเอกสาร
`() => …`	"**เมื่อถูกเรียก ให้ทำ…**"	บรรทัด 380 `onPress={() => openForm("borrow")}` = เมื่อกด ให้เปิดฟอร์มยืม	โน้ตแปะ "ถ้ามีคนกด ให้ทำสิ่งนี้"
`if (…) …`	ถ้า … ให้ทำ …	บรรทัด 202 `if (error) throw error;`	"ถ้าฝนตก ให้พกร่ม"
`await`	**รอ**ให้เสร็จก่อนค่อยไปบรรทัดถัดไป	บรรทัด 192 รออัปโหลดรูปเสร็จ	รอคิวโอนเงินผ่านก่อน

หน้าตา	อ่านว่า	ตัวอย่างจริง	เทียบกับ
`try {…} catch {…} throw`	ลองทำ ถ้าพังไปทำใน catch · throw = โยนปัญหาออกไป	บรรทัด 202 throw → กระโดดไปบรรทัด 208 catch	แผนสำรอง
`<ชื่อ ค่า={…}> … </ชื่อ>`	**ชิ้นส่วนบนหน้าจอ** (เขียนคล้าย HTML) · `ค่า={…}` = ตั้งค่าให้ชิ้นนั้น	บรรทัด 378–386 ปุ่ม `<TouchableOpacity disabled={atLimit}>` ข้างในมีไอคอนกับตัวหนังสือ	ตัวต่อเลโก้ ชิ้นซ้อนชิ้น

Method คืออะไร

Method (หรือ function) = **คำสั่งสำเร็จรูปที่มีชื่อ** เรียกชื่อแล้วตามด้วย `( )` มันก็ทำงานให้ เหมือนกดปุ่มไมโครเวฟ "อุ่นข้าว" ไม่ต้องรู้ว่าข้างในทำงานยังไง รู้แค่ว่ากดแล้วได้อะไร

- **มีจุดข้างหน้า** เช่น `supabase.rpc` = คำสั่ง `rpc` **ของ** `supabase` (มาจากเครื่องมือภายนอก)
- **ไม่มีจุด** เช่น `openForm` = ทีมเขียนเอง หาที่มาได้โดยค้น `const openForm` หรือ `function openForm`

Method 8 ตัวที่เจอใน 3 รูป

method	ทำอะไร	เจอที่บรรทัด	เขียนไว้ที่ไหน
`supabase.rpc("ชื่อ", {…})`	**โทรหาหลังบ้าน** เรียกบริการตามชื่อ พร้อมแนบฟอร์มข้อมูล	194, 197	เครื่องมือ Supabase
`uploadBorrowPhoto(รูป)`	ย่อรูปเหลือด้านยาว 1600px แล้ว**อัปโหลด** ได้ที่อยู่รูปกลับมา	192	lib/borrowPhotos.ts
`openForm("borrow")`	**เปิดฟอร์ม** ล้างค่าเก่า แล้วตั้งจำนวนวันเริ่มต้นให้	380	app/scan.tsx บรรทัด 155
`runLookup(รหัส)`	**ค้นชิ้นนี้ใหม่** ถามหลังบ้านว่าตอนนี้สถานะอะไร	207	app/scan.tsx บรรทัด 103
`notify("หัวข้อ", "ข้อความ")`	**เด้งป๊อปอัป**ที่มีปุ่ม "โอเค"	210	lib/notify.tsx

method	ทำอะไร	เจอที่บรรทัด	เขียนไว้ที่ไหน
`setConfirmOpen(false)`	เปลี่ยนค่าที่หน้าจอจำไว้ → **ปิดหน้าต่างยืนยัน**	203, 209	ชื่อที่ขึ้นต้นด้วย `set` = เปลี่ยนค่าที่จอจำไว้ แล้วจอวาดใหม่เอง (จับคู่กันที่บรรทัด 78 เป็นต้นไป)
`setJustSent(ข้อความ)`	ตั้งข้อความ "ส่งคำขอ ... แล้ว" ให้ขึ้นบนจอ	204	
`ข้อความ.trim()`	**ตัดช่องว่าง**หัวท้ายข้อความ (พิมพ์แต่เว้นวรรค = ถือว่ายังไม่ได้เขียน)	175	มีมากับภาษาเลย

ลองแยกทีละคำ 3 บรรทัด

วิธีนี้ใช้ได้กับทุกบรรทัด: **หั่นเป็นชิ้น → แปลทีละชิ้น → อ่านรวมเป็นประโยคไทย**

บรรทัด 363

```
const atLimit = activeCount !== null && activeCount >= max;
```

`const atLimit =`	ตั้งชื่อว่า "ถึงเพดาน" ให้เท่ากับ...
`activeCount !== null`	จำนวนที่ยืมอยู่ **ไม่ใช่**ค่าว่าง (รู้ตัวเลขแล้ว)
`&&`	และ
`activeCount >= max`	จำนวนที่ยืมอยู่ **มากกว่าหรือเท่ากับ** เพดาน

อ่านรวม: "ถึงเพดาน คือ รู้จำนวนที่ยืมแล้ว **และ** จำนวนนั้นไม่น้อยกว่าเพดาน"

บรรทัด 192

```
const photoPath = await uploadBorrowPhoto(photoUri);
```

`const photoPath =`	ตั้งชื่อว่า "ที่อยู่รูป" ให้เท่ากับ...
`await`	รอให้เสร็จก่อน
`uploadBorrowPhoto(…)`	สั่งอัปโหลดรูปหลักฐานการยืม
`photoUri`	ยื่นรูปที่เพิ่งถ่ายให้

อ่านรวม: "อัปโหลดรูปที่ถ่าย รอจนเสร็จ แล้วจดที่อยู่รูปไว้ในชื่อ photoPath"

บรรทัด 194–195

```
? await supabase.rpc("request_borrow", { p_item_id: item.id, p_days: days, … })
```

`?`	ถ้าเงื่อนไขข้างหน้าใช่ (กำลังยืม) ให้ทำส่วนนี้
`await`	รอให้เสร็จ
`supabase.rpc(…)`	โทรหาหลังบ้าน
`"request_borrow"`	ขอบริการชื่อ "ขอยืม"
`{ p_item_id: item.id, … }`	แนบฟอร์ม: ช่องรหัสอุปกรณ์ = id **ของ** ชิ้นนี้, ช่องจำนวนวัน = วันที่เลือก …

อ่านรวม: "ถ้ากำลังยืม ให้โทรหาหลังบ้าน ขอบริการขอยืม แนบรหัสอุปกรณ์กับจำนวนวัน แล้วรอคำตอบ"

10.2 ทริก: อ่านโค้ดที่ไม่มีคอมเมนต์

ถ้าอาจารย์เปิดโค้ดท่อนที่ไม่มีคอมเมนต์ภาษาไทยให้ดู ให้ใช้ 3 ท่อนนี้ซ้อมก่อน ทั้ง 3 ท่อนอยู่ใน**ไฟล์เดียวกัน** คือ `app/scan.tsx` (หน้าสแกนยืม-คืนที่จะเดโม) เรียงตามลำดับที่ผู้ใช้กด เป็น**โค้ดจริงไม่แก้ไข** เลขบรรทัดตรงกับไฟล์ ณ วันที่ 29 ต.ค. 2569

เปิดไฟล์ยังไง (ทำก่อนพรีเซนต์ เปิดแท็บค้างไว้เลย)

บนเว็บ GitHub
เข้า **github.com/Sineeyagron/Project1** → โฟลเดอร์ `iot-lab-management` → `app` → `scan.tsx` · หรือกดปุ่ม **t** บนหน้า repo แล้วพิมพ์ `scan.tsx` (ถ้า repo เป็นแบบส่วนตัว ต้องล็อกอินบัญชีที่มีสิทธิ์ก่อน)

ในโปรแกรม VS Code
เปิดโฟลเดอร์ `iot-lab-management` → กด **Ctrl+P** พิมพ์ `scan.tsx` → Enter → กด **Ctrl+G** พิมพ์เลขบรรทัด เช่น `171` → Enter

หาจากคำบนจอ (ทริกสำคัญ)
เห็นปุ่มบนจอเขียนว่าอะไร ค้นคำนั้นในโค้ด เช่น ค้น **"ขอยืมอุปกรณ์นี้"** จะเจอบรรทัด 385 ทันที · VS Code กด **Ctrl+Shift+F** · GitHub กด **Ctrl+F** ในหน้าไฟล์

ถ้าเลขบรรทัดไม่ตรง (มีคนแก้ไฟล์ทีหลัง): ให้ค้นด้วยชื่อแทน เช่น ค้น `atLimit`, `formReady`, `request_borrow` ก็เจอเหมือนกัน

กุญแจอ่านโค้ด: จำ 3 อย่างนี้พอ

1. **หาภาษาไทยในโค้ดก่อน** ข้อความไทยในโค้ดคือคำที่ขึ้นบนจอ เห็นคำไหนก็รู้ว่าโค้ดตรงนั้นคือส่วนไหนของหน้าจอ
2. **อ่านชื่อเป็นภาษาอังกฤษ** โปรแกรมเมอร์ตั้งชื่อตามความหมาย เช่น `atLimit` = ถึงเพดาน · `formReady` = ฟอร์มพร้อม · `photoUri` = รูป · `condition` = สภาพ · `submit` = ส่ง · ตัวที่ขึ้นต้นด้วย `set` = เปลี่ยนค่าให้หน้าจออัปเดต
3. **แปลสัญลักษณ์เป็นคำไทย** ตามตารางนี้

สัญลักษณ์	อ่านว่า	สัญลักษณ์	อ่านว่า
`===`	เท่ากับ	&&	และ (ต้องจริงทุกข้อ)
`!==`	ไม่เท่ากับ	||	หรือ (จริงข้อเดียวก็พอ)
`>=`	มากกว่าหรือเท่ากับ	!	ไม่ · `!!` = "มีค่าแล้วไหม"

สัญลักษณ์	อ่านว่า	สัญลักษณ์	อ่านว่า
`A ? B : C`	ถ้า A ใช่ ใช้ B ไม่งั้นใช้ C	??	ถ้าไม่มีค่า ใช้ค่านี้แทน
`await`	รอให้เสร็จก่อน	try / catch	ลองทำ ถ้าพลาดไปทำใน catch

รูปที่ 1: ปุ่ม "ขอยืมอุปกรณ์นี้" กดไม่ได้เมื่อยืมครบ

app/scan.tsx · บรรทัด 362–363 และ 378–386

```
362 const max = lookup.max_active_borrows ?? 3;
363 const atLimit = activeCount !== null && activeCount >= max;
    ⋮
378 <TouchableOpacity
379   style={[s.primaryBtn, atLimit && { opacity: 0.45 }]}
380   onPress={() => openForm("borrow")}
381   disabled={atLimit}
382   activeOpacity={0.85}
383 >
384   <Ionicons name="hand-left-outline" size={18} color="#fff" />
385   <Text style={s.primaryBtnText}>ขอยืมอุปกรณ์นี้</Text>
386 </TouchableOpacity>
```

385	เจอภาษาไทย "ขอยืมอุปกรณ์นี้" = **นี่คือปุ่มขอยืม**ในหน้าสแกน · บรรทัด 384 คือไอคอนรูปมือบนปุ่ม
378–386	`TouchableOpacity` = "ของที่แตะได้" ก็คือตัวปุ่ม ทุกอย่างระหว่าง `<TouchableOpacity` ถึง `</TouchableOpacity>` คือของที่อยู่บนปุ่ม
362	`max` = เพดานจำนวนยืม เอามาจากหลังบ้าน ถ้าไม่มีค่าใช้ 3 (`??`)
363	`atLimit` (ถึงเพดานไหม) = รู้จำนวนที่ยืมอยู่แล้ว **และ** จำนวนนั้น ≥ เพดาน
381	`disabled={atLimit}` = ถ้าถึงเพดาน **ปุ่มกดไม่ได้** · บรรทัด 379 ทำให้ปุ่มจางเหลือ 45%
380	`onPress` = กดแล้วทำอะไร → เปิดฟอร์มยืม

พูด ตรงนี้คือปุ่มขอยืมครับ/ค่ะ บรรทัด 363 เช็กว่ายืมครบเพดานหรือยัง ถ้าครบ บรรทัด 381 จะสั่งให้ปุ่มกดไม่ได้ และบรรทัด 379 ทำให้ปุ่มจาง ส่วนหลังบ้านก็เช็กซ้ำอีกชั้น

รูปที่ 2: ปุ่ม "ส่งคำขอ" กดได้เมื่อไร

app/scan.tsx · บรรทัด 171–175

```
171 const needsEvidence = mode === "borrow" || mode === "return" || mode === "claim";
172 const formReady =
173   !!mode &&
174   (mode === "return" || !!days) &&
175   (!needsEvidence || (!!condition && !!photoUri && (condition === "good" ||
!!note.trim())));
```

171	`needsEvidence` (ต้องมีหลักฐานไหม) = ใช่ ถ้ากำลัง**ยืม** หรือ**คืน** หรือ**รับของตามนัด** (ยืมต่อไม่ต้องถ่ายรูป)
172	`formReady` (ฟอร์มพร้อมส่งไหม) = ต้องผ่าน**ครบ 3 ข้อ** ต่อด้วย `&&`
173	ข้อ 1: เลือกแล้วว่าจะทำอะไร (ยืม/คืน/ยืมต่อ)
174	ข้อ 2: เป็นการคืน **หรือ** เลือกจำนวนวันแล้ว (คืนไม่ต้องเลือกวัน)
175	ข้อ 3: ไม่ต้องมีหลักฐาน **หรือ** (เลือกสภาพแล้ว **และ** ถ่ายรูปแล้ว **และ** (สภาพปกติ **หรือ** เขียนหมายเหตุแล้ว))

โค้ดที่ใช้ค่านี้: บรรทัด 535 `disabled={!formReady || submitting}` = ปุ่มส่งกดไม่ได้ถ้าฟอร์มยังไม่พร้อม หรือกำลังส่งอยู่ (กันกดซ้ำ)

พูด บรรทัด 175 คือเหตุผลที่ยืมโดยไม่ถ่ายรูปไม่ได้ครับ/ค่ะ ปุ่มส่งจะกดได้ก็ต่อเมื่อเลือกสภาพแล้ว ถ่ายรูปแล้ว และถ้าบอกว่าชำรุดต้องเขียนว่าชำรุดตรงไหน

รูปที่ 3: กดส่งแล้วเกิดอะไรขึ้น

app/scan.tsx · บรรทัด 191–210 (อยู่ในฟังก์ชัน submit ที่เริ่มบรรทัด 177)

```
191 } else {
192   const photoPath = await uploadBorrowPhoto(photoUri);
193   ({ error } = mode === "borrow"
194     ? await supabase.rpc("request_borrow", {
195         p_item_id: item.id, p_days: days, p_condition: condition, p_note: note,
p_photo_path: photoPath,
196       })
197     : await supabase.rpc("request_return", {
198         p_item_id: item.id, p_condition: condition, p_note: note, p_photo_path:
photoPath,
199       }));
200 }
201
202 if (error) throw error;
203 setConfirmOpen(false);
204 setJustSent(mode === "claim"
205   ? `รับ ${item.item_code} ตามนัดแล้ว บันทึกการยืมเรียบร้อย`
206   : `ส่งคำ${KIND_TH[mode]} ${item.item_code} แล้ว รอผู้ดูแลอนุมัติ`);
207 await runLookup(item.id);
208 } catch (e: any) {
209   setConfirmOpen(false);
210   notify("ส่งคำขอไม่สำเร็จ", e.message || "กรุณาลองใหม่อีกครั้ง");
```

192	**อัปโหลดรูป**ก่อน (`await` = รอจนเสร็จ) ได้ที่อยู่ของรูปกลับมา
193–199	ถ้ากำลัง**ยืม** (`?`) → ส่ง `request_borrow` · ไม่งั้น (`:`) คือ**คืน** → ส่ง `request_return` · ส่งไปแค่ ชิ้นไหน กี่วัน สภาพ หมายเหตุ รูป
202	ถ้าหลังบ้านตอบว่า**มีปัญหา** (เช่น ยืมครบ 3 ชิ้น) → กระโดดไปที่ `catch` บรรทัด 208
204–206	สำเร็จ → ขึ้นข้อความ "ส่งคำขอยืม … แล้ว รอผู้ดูแลอนุมัติ" · `${...}` = เอาค่ามาเติมในข้อความ เช่น รหัสอุปกรณ์
207	สแกนชิ้นเดิมซ้ำให้เอง หน้าจอจะเปลี่ยนเป็น "รออนุมัติ"
208–210	พลาด → เด้งป๊อปอัป "ส่งคำขอไม่สำเร็จ" พร้อมเหตุผลที่หลังบ้านส่งมา

พูด กดส่งแล้ว แอปอัปโหลดรูปก่อน แล้วส่งคำขอไปหลังบ้าน หลังบ้านเป็นคนตัดสิน ถ้าผ่านก็ขึ้นว่ารอผู้ดูแลอนุมัติ ถ้าไม่ผ่าน เช่น ยืมครบแล้ว ก็เด้งเหตุผลบอก

ซ้อมแบบนี้: เปิดไฟล์จริง กด Ctrl+G ไปบรรทัด 363, 175, 202 แล้วลองอธิบายให้เพื่อนฟังโดยไม่ดูคู่มือ · ถ้าอาจารย์ชี้ท่อนอื่น ใช้ 3 กุญแจข้างบน (หาภาษาไทย → อ่านชื่อ → แปลสัญลักษณ์) ยังไม่แน่ใจก็ตอบตรง ๆ ได้ อย่าเดา

10.3 ท่อนเสริม: กติกาหลังบ้าน + ไฟล์ตัวช่วย

อ่านเพิ่มถ้ามีเวลา 4 ท่อนนี้เป็นโค้ดจริงเหมือนกัน (ตัดบางบรรทัดให้สั้น และเติมคอมเมนต์ไทยเพื่ออธิบาย) ท่อนเสริม 1–2 เป็นภาษา SQL ฝั่งหลังบ้าน ใช้กติกาเดียวกับที่สอนใน 10.1 คือ `if` ... `raise exception` = ถ้า ... ให้ปฏิเสธ

ท่อนเสริม 1: หลังบ้านตัดสินโควตา 3 ชิ้น (ฝั่งฐานข้อมูล)

supabase/migrations/20261011200000_f2_pickup_hold_pending.sql · ฟังก์ชัน request_borrow

```
v_max    := public._setting_int('max_active_borrows', 3);  -- เพดาน (ค่าในหน้าตั้งค่า ถ้าไม่มีใช้ 3)
v_active := public._active_borrow_count(v_uid);           -- คนนี้ยืม/รออยู่กี่ชิ้น
if v_active >= v_max then
  raise exception 'ยืมพร้อมกันได้สูงสุด % ชิ้น ...', v_max, v_active;
end if;
```

- **พูดว่า:** "หลังบ้านอ่านเพดานจากการตั้งค่า นับว่าคนนี้ยืมอยู่กี่ชิ้น ถ้าครบแล้วก็ปฏิเสธพร้อมข้อความเตือน คำขอนั้นไม่ถูกบันทึกเลย"
- นี่คือเหตุผลที่ **"แก้แอปก็โกงไม่ได้"** เพราะการนับเกิดที่หลังบ้าน ไม่ได้อยู่ในมือถือ

ท่อนเสริม 2: กันแจ้งปัญหาห้องคอมรัว ๆ (แบบเดียวกับท่อนเสริม 1)

supabase/migrations/20261005120000_room_r3_reports.sql · ฟังก์ชัน report_room_problem

```
if v_today >= 5 then
  raise exception 'วันนี้แจ้งครบ 5 ครั้งแล้ว ลองใหม่พรุ่งนี้ หรือแจ้งผู้ดูแลในห้อง';
end if;
```

- **พูดว่า:** "ถ้าวันนี้คนนี้แจ้งไปแล้ว 5 ครั้ง ระบบไม่รับเพิ่ม กันแจ้งมั่ว" · **สังเกต:** กติกาทุกข้อในระบบเขียนแบบเดียวกัน คือ `if` (ถ้า) ... `raise exception` (ปฏิเสธ)

ท่อนเสริม 3: ชื่อและสีของสถานะ (ฝั่งแอป)

lib/status.ts

```
export const ITEM_STATUS = {
  available: { label: "ว่าง",     color: "#16a34a", ... },   // เขียว
  borrowed:  { label: "ถูกยืม",   color: "#b45309", ... },   // ส้ม
  repair:    { label: "ซ่อมบำรุง", color: "#dc2626", ... },   // แดง
};
```

- **พูดว่า:** "ฐานข้อมูลเก็บสถานะเป็นภาษาอังกฤษ เช่น available ไฟล์นี้เป็นพจนานุกรมแปลเป็นภาษาไทยและกำหนดสี ทุกหน้าของระบบยืม-คืนใช้ไฟล์เดียวกัน สีกับคำเลยตรงกันทุกหน้า แก้ที่เดียวเปลี่ยนทุกหน้า" (ระบบห้องคอมมีไฟล์ของตัวเองแยกไว้ที่ `lib/room/roomStatus.ts`)

ท่อนเสริม 4: TA เข้าหน้าไหนได้บ้าง (ฝั่งแอป)

lib/roles.ts · ย่อให้อ่านง่าย

```
function canAccess(role, path) {
  if (role === "admin") return true;     // Admin เข้าได้ทุกหน้า
  if (role !== "ta") return false;      // นักศึกษาเข้าหน้าผู้ดูแลไม่ได้
  return TA_ROUTES.has(path);             // TA เข้าได้เฉพาะหน้าที่อยู่ในรายชื่อ
}
```

- **พูดว่า:** "ฟังก์ชันนี้ใช้ซ่อนหน้าที่ไม่มีสิทธิ์ในแอป Admin เข้าได้หมด TA เข้าได้เฉพาะหน้าที่อยู่ในรายชื่อ `TA_ROUTES` ส่วนสิทธิ์จริงฐานข้อมูลตรวจซ้ำอีกชั้น"

ถ้าโดนชี้โค้ดที่ไม่รู้จัก: (1) ดูชื่อไฟล์ว่าอยู่โฟลเดอร์ไหน จะรู้ว่าเป็นหน้าจอ ตัวช่วย หรือกติกา (2) อ่านคอมเมนต์ภาษาไทยเหนือท่อนนั้น (3) ถ้ายังไม่แน่ใจ ตอบตรง ๆ ได้ว่า "ส่วนนี้ขอให้คนเขียนอธิบายเพิ่มนะครับ/ค่ะ" อย่าเดา

11 คำศัพท์ที่อาจได้ยิน

คำ	ความหมายแบบง่าย
Frontend / หน้าบ้าน	ส่วนที่ผู้ใช้เห็นและกด (ตัวแอป)
Backend / หลังบ้าน	ส่วนที่เก็บข้อมูลและประมวลผลอยู่บนเซิร์ฟเวอร์ ผู้ใช้มองไม่เห็น
Database / ฐานข้อมูล	ที่เก็บข้อมูลเป็นตาราง คล้าย Excel หลายแผ่นที่เชื่อมกัน
Table / ตาราง	ข้อมูลเรื่องเดียวกัน เช่น ตารางอุปกรณ์ ตารางการยืม
API / RPC	"ช่องบริการ" ที่แอปเรียกให้หลังบ้านทำงาน เช่น request_borrow = ส่งคำขอยืม
RLS (Row Level Security)	กฎสิทธิ์รายแถว ใครเห็น/แก้ข้อมูลแถวไหนได้
Authentication / Auth	การยืนยันตัวตน (ล็อกอิน)
Realtime	ข้อมูลอัปเดตบนจอทันทีโดยไม่ต้องรีเฟรช
Cron job	งานที่ตั้งเวลาให้ระบบทำเองซ้ำ ๆ เช่น ทุกวัน 08:00
PWA	เว็บที่ติดตั้งเป็นไอคอนบนมือถือและทำงานเหมือนแอป
Push Notification	แจ้งเตือนที่เด้งขึ้นบนเครื่อง แม้ไม่ได้เปิดแอป
Deploy	เอาโค้ดเวอร์ชันใหม่ขึ้นไปให้ผู้ใช้ใช้จริง
Git / GitHub / Commit / Pull Request	ระบบเก็บประวัติโค้ด · commit = บันทึกการแก้ 1 ครั้ง · Pull Request = ขอรวมงานเข้าโค้ดหลัก

คำ	ความหมายแบบง่าย
Use Case Diagram	แผนภาพว่าผู้ใช้แต่ละบทบาททำอะไรกับระบบได้บ้าง
ER Diagram	แผนภาพตารางในฐานข้อมูล และตารางไหนเชื่อมกับตารางไหน
Mockup	แบบร่างหน้าจอ ทำให้ดูก่อนเขียนโค้ดจริง